linaro™

Arm Solutions at Lightspeed

Between Building and Testing Your Linux Driver

Open Source Summit Europe 2025

Krzysztof Kozlowski, Linaro
krzk@kernel.org, @krzk@social.kernel.org

Introduction

- Krzysztof Kozlowski
- I work for Linaro
- I am the co-maintainer of few subsystems in the Linux kernel
- I sent a lot of patches for the Linux kernel, often as result of static analysis
  - I read even more patches

Agenda

- Why building your Linux driver is not enough?
- Clang / LLVM
- Sparse, Smatch, Coccinelle
- Finding bugs in existing code
- LLM aka AI Slop

Why Building Your Linux Driver Is Not Enough?

Why Building Is Not Enough?

- TLDR:
  - Better patch
  - Happier maintainers
  - Faster review / acceptance of the code

What is Static Analysis?

- *“Static program analysis is the analysis of computer programs performed without executing them…”*
  - Source: https://en.wikipedia.org/wiki/Static_program_analysis
- Compilers check syntax correctness to produce a binary
  - You can write broken code which will still compile
  - Both GCC and Clang already have some logic to detect code flow issues
- Static analyzers follow the code flow and logic behind
  - At least that’s the goal…
- There are several tools for static analysis but…
  - Not all are Open-Source
  - Not all handle Linux kernel well
  - Talk focuses on tools fulfilling both aspects above

Why Building Is Not Enough?

```
static int __send_command(struct brcmstb_dpfe_priv *priv, unsigned int cmd, u32 result[])
{
        const u32 *msg = priv->dpfe_api->command[cmd];
        int ret = 0;

        if (cmd >= DPFE_CMD_MAX)
                return -1;

        ...
}
```

Why Building Is Not Enough?

```
static int __send_command(struct brcmstb_dpfe_priv *priv, unsigned int cmd, u32 result[])
{
        const u32 *msg = priv->dpfe_api->command[cmd];
        int ret = 0;

        if (cmd >= DPFE_CMD_MAX)
                return -1;

        ...
}

...

struct dpfe_api {
        int version;
        const char *fw_name;
        const struct attribute_group **sysfs_attrs;
        u32 command[DPFE_CMD_MAX][MSG_FIELD_MAX];
};
```

Why Building Is Not Enough?

```
static int __send_command(struct brcmstb_dpfe_priv *priv, unsigned int cmd, u32 result[])
{
        const u32 *msg = priv->dpfe_api->command[cmd];
        int ret = 0;

        if (cmd >= DPFE_CMD_MAX)
                return -1;

        ...
}

...

struct dpfe_api {
        int version;
        const char *fw_name;
        const struct attribute_group **sysfs_attrs;
        u32 command[DPFE_CMD_MAX][MSG_FIELD_MAX];
};
```

Where Is the Bug

```
        const u32 *msg = priv->dpfe_api->command[cmd];

        if (cmd >= DPFE_CMD_MAX)
                return -1;
```

- Accessing ‘command[cmd]’ before validating [cmd] index is:
  - Confusing from readability point of view
  - Not correct: out of bounds access if: ‘cmd is >= DPFE_CMD_MAX’
- Smatch (more on that soon):
  - drivers/memory/brcmstb_dpfe.c:442 __send_command() error: testing array offset 'cmd' after use.

Open-Source Static Analysis to the Rescue!


{</>}
SUPER
OPEN SOURCE

Clang / LLVM

Clang / LLVM

- Clang/LLVM has some additional checks comparing to GCC
- It is always worth to build your code with it
- Use W=1 for additional warnings
- TLDR:

Source: https://llvm.org/Logo.html

```
$ make CC="ccache clang-20" LLVM=-20 W=1 <TARGET>
$ make CC="ccache clang-20" ARCH=arm64 KBUILD_OUTPUT=build/ LLVM=-20 W=1 -j12 drivers/
```

- Replace ‘20’ with proper llvm version you have
- It’s easy!
- And cross compiling is even easier - no need for separate cross compilers

Clang and cleanup.h

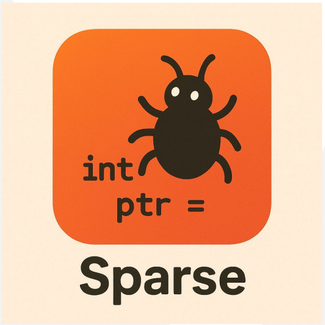
- You must compile your code with clang if you use any scope-based cleanups or guards (include/linux/cleanup.h)
  - __free()
  - guard()
  - It is also a must if you touch existing code with cleanups
- GCC does not report jumps bypassing initialization
- Clang does:

```
error: cannot jump from this goto statement to its label
  105 |                 goto err;
      |                 ^
note: jump bypasses initialization of variable with __attribute__((cleanup))
  106 |         guard(mutex)(&pg->state_lock);
      |         ^
```

Sparse

Sparse

- Started by Linus Torvalds, recently mostly developed and maintained by Luc Van Oostenryck
  - https://git.kernel.org/pub/scm/devel/sparse/sparse.git
- Comes with several checks specific to Linux kernel
- See Documentation/dev-tools/sparse.rst for more
- TLDR:



Fake logo :)

```
# Run on only recompiled files (C=1):
$ make C=1 CHECK="~/path/to/sparse/sparse" -j12 <TARGET>
$ make C=1 CHECK="~/path/to/sparse/sparse" -j12 drivers/

# Run on all files (C=2):
$ make C=2 CHECK="~/path/to/sparse/sparse" -j12 drivers/

# And if you cross compile:
$ CROSS_COMPILE="ccache aarch64-linux-gnu-" ARCH=arm64 KBUILD_OUTPUT=build/ \
    make C=1 CHECK="~/path/to/sparse/sparse" -j12 drivers/
```

Sparse and Locking

- Sparse relies on C annotations to statically determine locking flow
  - Possible missing synchronization (forgot to lock() when calling some function)
  - Possible deadlock (forgot to unlock() on exit)
- Functions should use __must_hold / __acquires / __releases so context tracking would work
  - `warning: context imbalance in 'iwl_mvm_start_get_nvm' - wrong count at exit`
- Real bugs, e.g. commit 97c0979d0d72 ("iwlwifi: mvm: fix imbalanced locking in iwl_mvm_start_get_nvm()")

Sparse Typical Warnings

- `symbol 'yoga_c630_ucsi_ops' was not declared. Should it be static?`
  - Usually missing `static` keyword
- `incorrect type in assignment (different address spaces)`
- `incorrect type in argument 1 (different address spaces)`
- `subtraction of different types can't work (different address spaces)`
  - Mixing user / kernel pointers
    - Serious bug leading to security vulnerabilities
  - Mixing kernel / MMIO (IOMEM) pointers
    - Code probably works fine for given platform but is not portable
    - Usually difficult to fix later (e.g. requires hardware and likely code is spaghetti), so please get it right at the first try

Sparse Typical Warnings

- `warning: cast to restricted __le16`
- `warning: incorrect type in assignment (different base types)`
  - Mixing endianness between types, e.g. via missing endianness annotations (e.g. __le16) or conversions (le16_to_cpu())
  - Code probably works fine for given platform but is not portable
    - ARM is bi-endian, although little-endian in practice, thus issue is unlikely to be observed
  - Usually difficult to fix later (e.g. requires hardware), so please get it right at the first try
  - Nice brain exercise: commit 7b515f35a911 ("net: enetc: Correct endianness handling in _enetc_rd_reg64")

Smatch

Smatch

- Smatch is a great tool developed by Dan Carpenter for both kernel and userland C code
  - https://github.com/error27/smatch
- See Documentation/smatch.rst how to build it (make :))
- Just like sparse, you can just build and use as CHECK tool
- TLDR:

Fake logo :)

```
$ ~/path/to/smatch/smatch_scripts/kchecker drivers/

# Or via kernel CHECK style from previous slides (same C=1 and C=2 difference).
# Remember to set also ARCH, CROSS_COMPILE and KBUILD_OUTPUT depending on your setup:
$ make C=1 CHECK="~/path/to/smatch/smatch -p=kernel" -j12 drivers/
```

Smatch

- Smatch really tries to guess what is the intention behind the code
  - Even if there is no bug, it leads to more readable code
- Leaking resources
  - Not only memory, but also types like clocks, IO mappings
- Uninitialized local variables
- Possible NULL pointer dereferences
- Confusing indentation (and possible incorrect code)

Smatch and Assumptions

```
const struct foo *ddata = of_device_get_match_data(dev);

if (ddata) {
        // Do something
        ...
}

...

if (ddata->quirks & CQSPI_SUPPORT_DEVICE_RESET) {
        // Handle quirks
        ...
}
```

Smatch and Assumptions

```
const struct foo *ddata = of_device_get_match_data(dev);

if (ddata) {
        // Do something
        ...
}

...

if (ddata->quirks & CQSPI_SUPPORT_DEVICE_RESET) {
        // Handle quirks
        ...
}
```

‘ddata’ can be NULL, apparently

Unconditional dereference of ‘ddata’

- Code is either incorrect or confusing
- Smatch will point it out:
  - drivers/spi/spi-cadence-quadspi.c:1942 cqspi_probe() error: we previously assumed 'ddata' could be null (see line 1885)

Smatch and False Positives

- Since Smatch is trying to find intention behind the code, it might got it wrong
- Linux kernel has plenty of clever code
  - Hopefully intentionally clever code
- However if you write drivers, you should write readable and maintainable code
  - Clever does not mean readable
- Many issues pointed out by Smatch are actually confusing code
  - Example commit b828b4bf29d1 ("ceph: fix variable dereferenced before check in ceph_umount_begin()")
- Carefully analyze **how Smatch warning should be fixed**, instead of just coding whatever makes the warning disappear

Coccinelle

Coccinelle

- Coccinelle is a tool for pattern matching and text transformation (semantic patches)
  - http://coccinelle.lip6.fr/
  - https://www.kernel.org/doc/html/latest/dev-tools/coccinelle.html
- Not really static analysis
- Heavily used in the Linux kernel for common tasks, common errors and mass code transformations
- Collection of kernel-relevant transformations:
  - scripts/coccinelle/
- apt-get install coccinelle

Source: https://coccinelle.gitlabpages.inria.fr/website/

Running Coccinelle

- Two modes of work: report and patch the code

```
# Standard report mode:
$ make coccicheck M=drivers/
# Or patch the source (not supported by all of scripts)
$ make coccicheck M=drivers/ MODE=patch
```

- Making output manageable (only code related warnings)

```
$ make coccicheck M=drivers/ DEBUG_FILE=cocci.err
```

- Testing specific drivers (Makefile targets)

```
$ make C=1 CHECK=scripts/coccicheck drivers/bluetooth/bfusb.o

# If you use KBUILD_OUTPUT=build/ variable, then coccicheck path needs fix:
$ make C=1 CHECK=../scripts/coccicheck drivers/bluetooth/bfusb.o
```

Running Specific Semanting Patch

- Using one semantic patch from scripts/coccinelle/

```
$ make coccicheck M=drivers/ COCCI=scripts/coccinelle/api/platform_get_irq.cocci

# If you use KBUILD_OUTPUT=build/ variable, then coccicheck path needs fix:
$ make coccicheck M=drivers/ COCCI=../scripts/coccinelle/api/platform_get_irq.cocci
```

Upstreaming 10 YO Code

- Linux kernel changed a lot during the last 10 years
  - 2015: Linux kernel v4.0
- We see drivers of v4.0-age and older being posted to mailing list
- How do we recognize them?
  - Repeat issues fixed during last 10 years
  - Follow old coding patterns
    - No dev_err_probe()
    - No devm-interfaces
    - Unneeded error message prints
    - Missing of_node_put()
    - Not handling !OF or !ACPI (avoid: of_match_ptr() and ACPI_PTR())
    - Any many other trivialities…
  - Shortly: old coding style
    - Might be hiding real issues, which we fixed long time ago

Upstreaming 10 YO Code

- My favorite litmus test, 99.9% accuracy of ~12 year old code:

```
static struct i2c_driver rt5133_driver = {
        .driver = {
                .name = "rt5133",
                .owner = THIS_MODULE,
                .of_match_table = of_match_ptr(rt5133_ofid_tbls),
        },
        .probe = rt5133_probe,
};
module_i2c_driver(rt5133_driver);
```

- What's wrong here?
  - Author did not run coccinelle… :)
    - Actually author should start with a new driver based on recently reviewed one
  - Independent of coccinelle: most likely missing compile test for other arch's (probably unnecessary of_match_ptr())

Transforming the Kernel

- Coccinelle is used heavily to transform the kernel
  - Old style -> new style
- If you have old driver, tools should help you to modernize it
  - Don’t use reviewers (people) instead of tools

Finding Bugs in Existing Code

Finding Bugs in Existing Code

- Use all these tools when developing new Linux drivers
- What about using them to check existing Linux kernel code?
  - Go ahead!
  - I and many others did it many times
- But:
  - **Always fix the cause, not the warning**
    - We do not want the warning to disappear, we want the actual issue to be fixed
    - Find the root cause
    - Maybe false positive?
    - Test your code
    - **Include in the commit msg the tool you used and trimmed text of the warning**
  - Not all Coccinelle fixups are welcomed by every maintainer
    - Many good Coccinelle scripts but also some not that useful

LLM (aka AI Slop)

- Using AI tool while coding is one thing…
- But do not send patches or bug reports based on output of LLM/AI tools
  - Not even close to static analysis
  - Unless you are a very experienced Linux kernel developer
- Wasting maintainers' time
  - Inaccurate or even fake findings
  - Hallucinated facts in responses to review
    - Wasting more of maintainers' time
  - Straightforward way to become ignored on the mailing lists
    - Because it was wasting time of maintainers (did I stress that enough?)
- If you ever do that regardless of my warnings, please **mention that in the commit message**
  - More details might emerge after Maintainers Summit 2025

linaro.org

Thank you